



LightUP
Your Potential

CHƯƠNG TRÌNH ĐÀO TẠO DOMAIN BACKEND

Lesson 3

Query Optimization



Biên soạn bởi Hội đồng chuyên môn - Domain Backend



GIẢNG VIÊN



Phan Hồng Linh

Project Manager

Data & Solution Architect Department

BEFORE START

Welcome to the hell!

- Tối ưu truy vấn là một bài toán khó và quan trọng. Hãy kiên nhẫn!

Bài giảng chứa hàm lượng kiến thức lớn

- Học kỹ trước khi làm bài test.
- Xem lại video, tìm hiểu thêm keywords nếu chưa hiểu rõ.

Slide chứa nhiều ví dụ code, SQL, JSON

- Download slide trước khi vào buổi học: <https://shorturl.at/wBD35>
- Đây là một môn học thực hành. Đừng chỉ đọc lý thuyết!

Nội dung bài giảng tập trung tối ưu truy vấn MySQL

- Nếu không nói gì thêm, mặc định hiểu phạm vi kiến thức thuộc Database *MySQL*, Storage Engine *InnoDB*, Index *B-tree*

BEFORE START



Optimize Database Level

- Storage engine: InnoDB, TokuDB, MyISAM, Memory,...
- Table structure: Index, Partition, Data Type, Schema Design,...
- Config (buffer pool, log file size, flush transaction commit, caching, binlog, row format,...)

Optimize OS/Hardware Level

- Disk (HDD, SSD, NVMe,...)
- CPU, Mem,...
- OS, Page size, IOPS, QPS, RAID,...

**WE ARE
HERE!**

Optimize Application Level

- *Query Optimization (SELECT, INSERT, UPDATE, DELETE, TRUNCATE,...)*
- Transaction, Locking, ORM, Connection Pool,...

MỤC LỤC



01. Giải phẫu database

02. Chẩn đoán slow query

03. Kế đơn

1

GIẢI PHẪU DATABASE



**Cấu trúc bên trong
database**

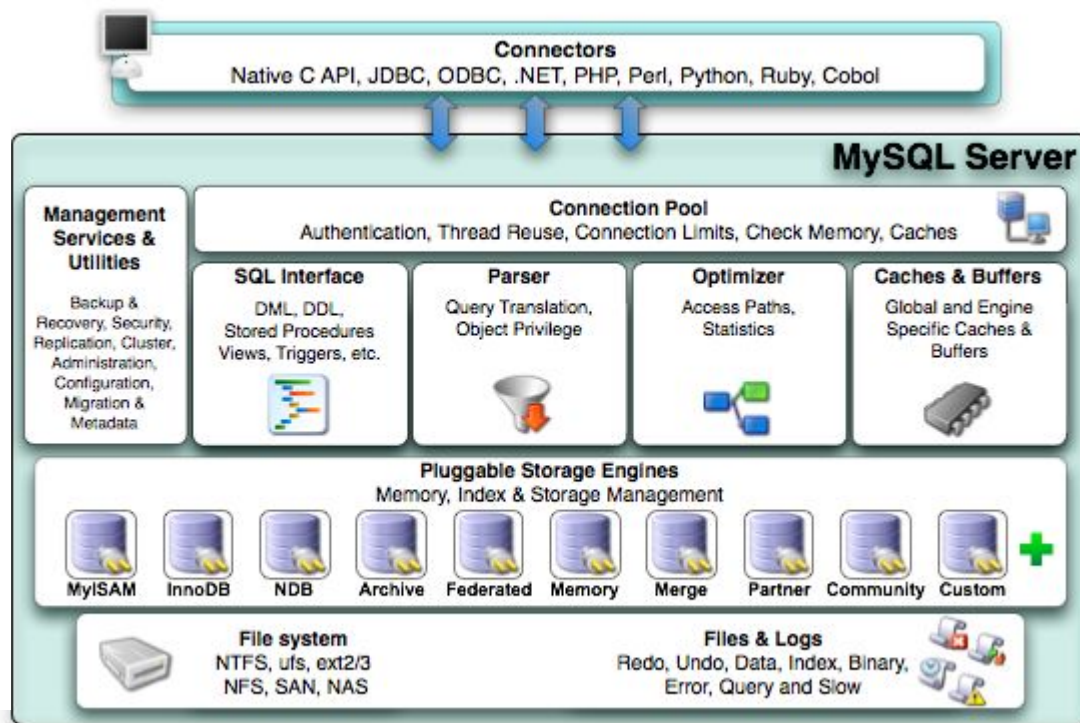
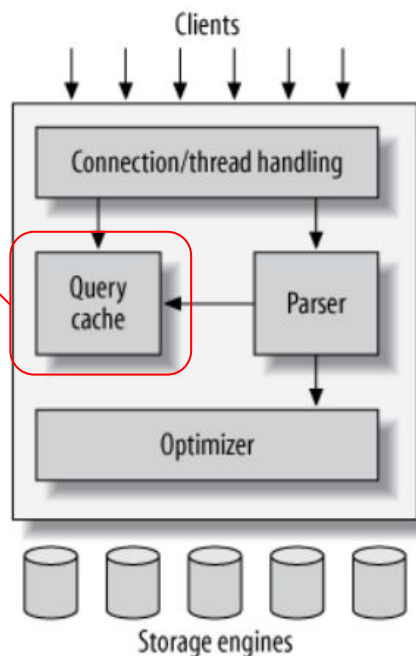


**Các thành phần tham gia
xử lý câu truy vấn**



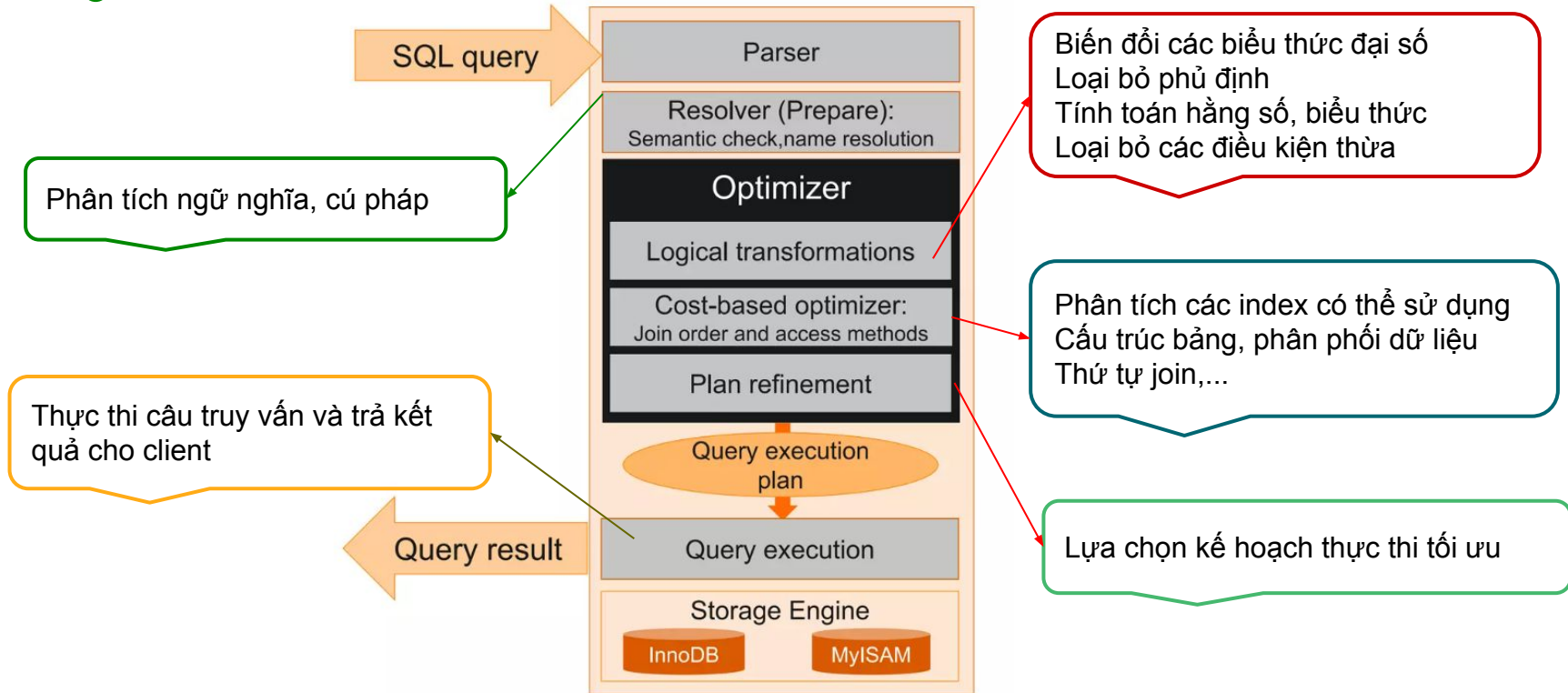
Giải phẫu database

Removed in
MySQL 8.0





X-ray vision





Logical Transformations

SELECT * FROM t1, t2 WHERE

t1.a = t2.a AND t2.a = 9 AND (NOT (t1.a > 10 OR t2.b > 3) OR (t1.b = t2.b + 7 AND t2.b = 5));

Negation
elimination

t1.a = t2.a AND t2.a = 9 AND (t1.a <= 10 AND t2.b <= 3 OR (t1.b = t2.b + 7 AND t2.b = 5));

Equality/const
propagation

t1.a = 9 AND t2.a = 9 AND (9 <= 10 AND t2.b <= 3 OR (t1.b = 5 + 7 AND t2.b = 5));

Evaluate const
expressions

t1.a = 9 AND t2.a = 9 AND (9 <= 10 AND t2.b <= 3 OR (t1.b = 12 AND t2.b = 5));

Trivial condition
removal

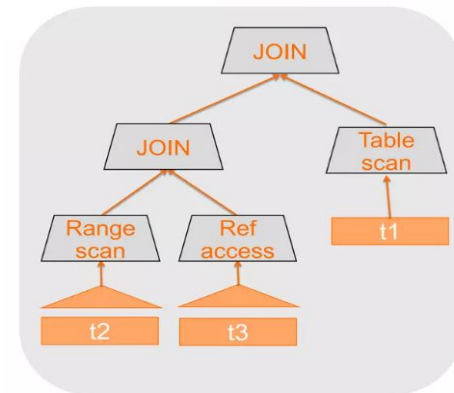
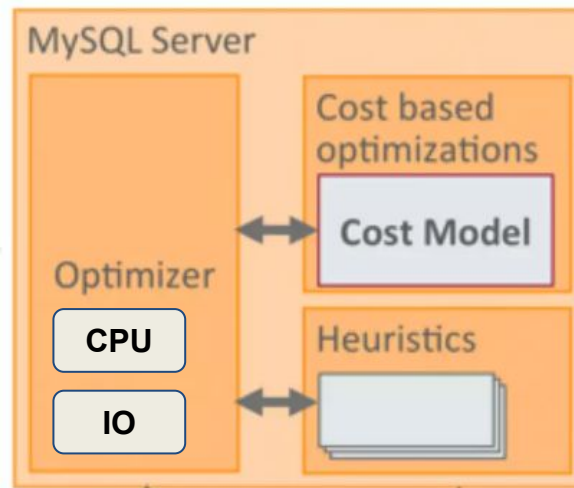
=TRUE

t1.a = 9 AND t2.a = 9 AND (t2.b <= 3 OR (t1.b = 12 AND t2.b = 5));



Cost-based Optimizer

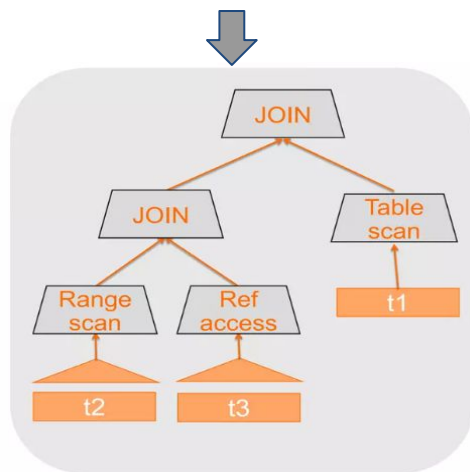
```
SELECT t1.a, t2.z  
FROM t1  
INNER JOIN t2 ON t1.a  
= t2.b  
INNER JOIN t3 ON t3.c  
= t2.d  
WHERE t1.x > 10  
AND t2.y = 100
```





Tổng kết giải phẫu database

```
SELECT t1.a, t2.z
FROM t1
INNER JOIN t2 ON t1.a = t2.b
INNER JOIN t3 ON t3.c = t2.d
WHERE t1.x > 10
AND t2.y = 100
```



SQL - Structured Query Language

- Là ngôn ngữ mô tả **mong muốn** của dev
- Không mô tả **kế hoạch thực thi** câu truy vấn

Câu truy vấn phải trải qua nhiều bước phân tích, biến đổi và tối ưu mới tiến hành thực thi.

- Thứ tự thực hiện câu truy vấn trong database có thể khác với thứ tự mô tả trong SQL.

Quá trình database lựa chọn kế hoạch thực thi tương tự quá trình Google Map chọn ra con đường phù hợp để tối lộ trình.

- Kế hoạch thực thi đưa ra từ Optimizer không phải bao giờ cũng chính xác.

2

CHẨN ĐOÁN SLOW QUERY



Query Cost



Explain



Optimizer Trace



Chẩn đoán lâm sàng: Query Cost

Slow queries # 2534757

Group: [QS](#)

User: [admin_report_prod](#)

Database: [ecom-db-admin](#)

Schema: ghtk

Host name: hn-fnx-prod-admin-dbreport-98122

Host ip: 0.0.0.0

Last err no: 0

Killed: 0

Query time:
261.533

Lock time:
0.000301

Rows sent: 20

Rows examined:
58870764

Rows affected: 0

Bytes sent: 884

Request id: 6cdfbc2b202b31ec9e946b5404010a7e

Query pattern: [#11928](#)



```
/* 6cdfbc2b202b31ec9e946b5404010a7e */  
SELECT Bag.id FROM bags AS `Bag` INNER  
JOIN `ghtk`.`transfers` AS `Transfer` ON  
(`Transfer`.`bag_order` = `Bag`.`order`)  
WHERE `Bag`.`is pkg` = 0 AND
```



Query run time

- Đơn giản.
- Khó bắt đúng bệnh và chữa tận gốc vấn đề.



Row Sent, Row Examined

- Tối ưu được phần lớn các câu truy vấn.
- Chưa hiểu đúng bản chất kế hoạch thực thi.



Cost-based Optimization

- Hiểu đúng bản chất Database thực thi truy vấn
- Khá khó hiểu với người mới.



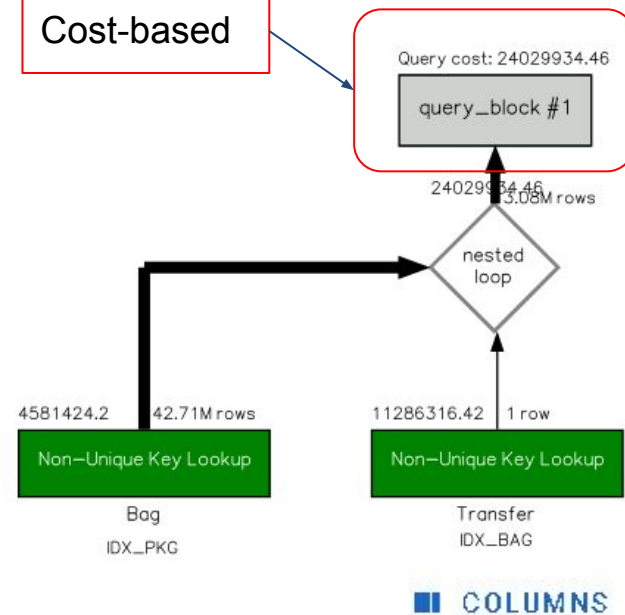
Query Cost

```
SELECT Bag.id FROM bags AS `Bag` INNER JOIN
`ghhk`.`transfers` AS `Transfer` ON
(`Transfer`.`bag_order` = `Bag`.`order`) WHERE
`Bag`.`is_pkg` = 0 AND `Transfer`.`exp_time` >=
'2023-08-01 16:00:00' AND `Transfer`.`exp_time` <=
'2023-09-01 16:59:00' AND `Bag`.`created` >=
'2023-08-05' LIMIT 20;
```

Slow queries

ID	Query time	Rows sent	Rows examined	Request id	Created
2534757	261.533	20	58870764	6cdfbc2b202b31ec9e946b5404010a7e	2023-09-05 14:04:14

Cost-based



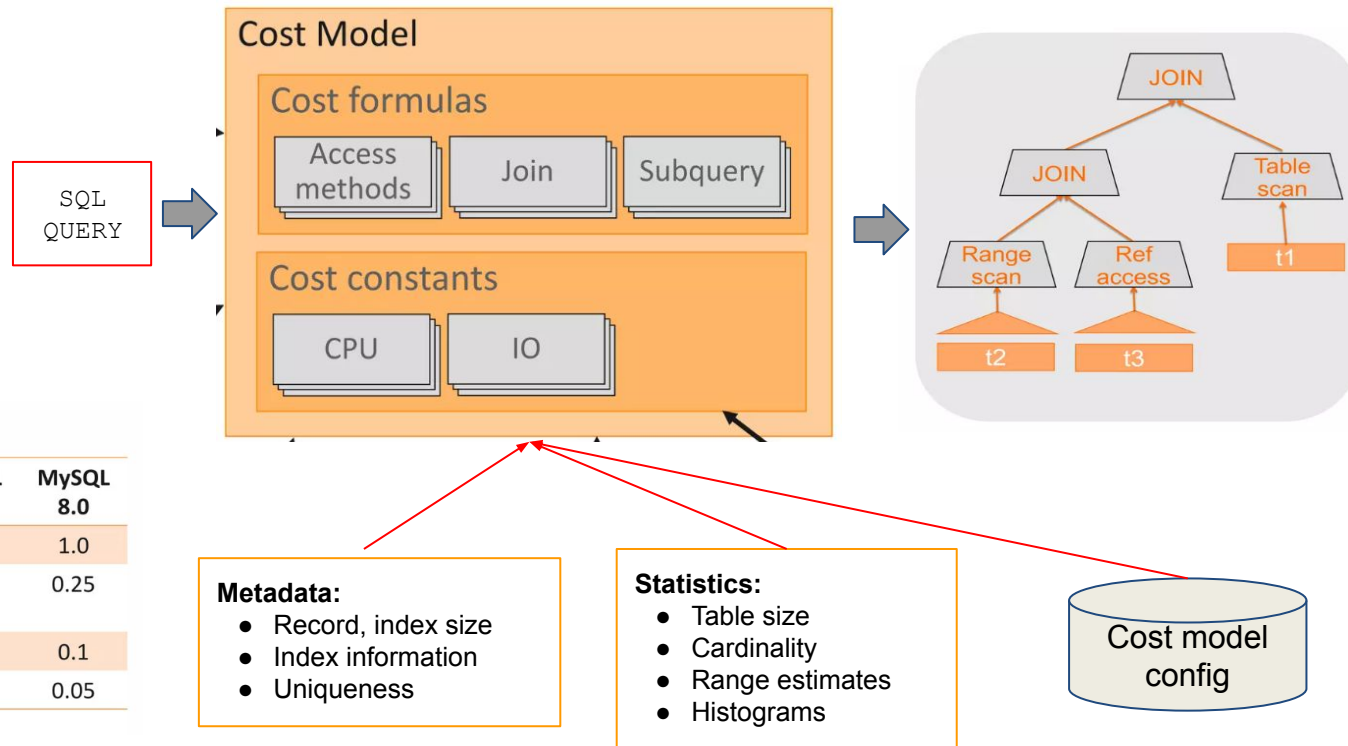
Query Cost

Cost-based:

- Index access
- Table scan
- Index scan, range scan, ref access
- Join order
- Subquery
- ...

Main cost constants

Cost	MySQL 5.7	MySQL 8.0
Read a random disk page	1.0	1.0
Read a data page from memory buffer	1.0	0.25
Evaluate query condition	0.2	0.1
Compare keys/records	0.1	0.05





Khám lâm sàng

```
CREATE TABLE `package_addresses` (  
  `id` BIGINT(20) NOT NULL AUTO INCREMENT,  
  `package_order` BIGINT(20) NOT NULL,  
  `type` ENUM('pickup','deliver','return')  
  NOT NULL,  
  `tel` VARCHAR(50) DEFAULT NULL,  
  `first_address` VARCHAR(500) DEFAULT NULL,  
  `cart_id` CHAR(36) DEFAULT NULL COMMENT 'id  
rổ lấy/giao/trả hàng',  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `UNIQU` (`package_order`,`type`)  
) ENGINE=INNODB DEFAULT CHARSET=utf8mb4  
COLLATE=utf8mb4_unicode_ci;
```

```
CREATE TABLE `carts` (  
  `id` CHAR(36) NOT NULL,  
  `type` TINYINT(4) NOT NULL COMMENT '1:  
picking, 2: delivering, 3: returing',  
  `alias` VARCHAR(100) NOT NULL,  
  `order` BIGINT UNSIGNED DEFAULT NULL,  
  `first_cod_id` CHAR(36) NOT NULL,  
  PRIMARY KEY (`id`),  
  KEY `IDX_UNIQUE` (`order`),  
) ENGINE=INNODB DEFAULT CHARSET=utf8  
COLLATE=utf8_unicode_ci;
```

```
SELECT pa.`package_order`, pa.`first_address`,  
c.`alias`, c.`first_cod_id`  
FROM `package_addresses` pa  
INNER JOIN carts c ON c.id = pa.cart_id  
WHERE pa.package_order = 1510775540 ;
```

Triệu chứng:

- Câu query join trên khóa chính
- Điều kiện WHERE trên INDEX
- Bị slow query chưa rõ nguyên nhân.
- Row examined: 112465



Chuyên khoa 1: EXPLAIN

```
EXPLAIN | DESCRIBE | DESC
[explain_type]
{explainable_stmt | FOR CONNECTION
connection_id}
```

EXPLAIN

```
SELECT pa.`package_order`, pa.`first_address`,
c.`alias`, c.`first_cod_id`
FROM `package_addresses` pa
INNER JOIN carts c ON c.id = pa.cart_id
WHERE pa.package_order = 1510775540 ;
```

```
explain_type:
    FORMAT =
TRADITIONAL | JSON | TREE

explainable_stmt:
    SELECT statement
    TABLE statement
    DELETE statement
    INSERT statement
    REPLACE statement
    UPDATE statement
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	pa	(NULL)	ref	UNIQ	UNIQ	8	const	3	100.00	(NULL)
1	SIMPLE	c	(NULL)	ALL	(NULL)	(NULL)	(NULL)	(NULL)	115228	100.00	Using where; Using join buffer (Block Nested Loop)

Read more about EXPLAIN [here](#)



EXPLAIN

id định danh
SELECT

Tên bảng

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	pa	(NULL)	ref	UNIQ	UNIQ	8	const	3	100.00	(NULL)
1	SIMPLE	c	(NULL)	ALL	(NULL)	(NULL)	(NULL)	(NULL)	115228	100.00	Using where; Using join buffer (Block Nested Loop)

select_type: Loại select dùng trong câu truy vấn

- **SIMPLE:** Là một câu SELECT cơ bản, không có subqueries, UNION,...
- **PRIMARY:** Là câu SELECT ngoài cùng của một phép JOIN (Outermost SELECT)
- **DERIVED:** Query nằm bên trong FROM
- **SUBQUERY:** Query đầu tiên trong subquery, không phụ thuộc vào query khác. Query này được execute 1 lần và cache lại.
- **DEPENDENT SUBQUERY:** Subquery phụ thuộc vào query nằm ngoài nó.
- **UNCACHEABLE SUBQUERY:** Query không thể cache lại được.
- **UNION:** Query là câu SELECT thứ hai của lệnh UNION
- **DEPENDENT UNION:** Trong subquery có union, và subquery đó thuộc loại DEPENDENT SUBQUERY
- **UNCACHEABLE UNION:** Câu subquery thứ 2 (hoặc thứ n) của UNION thuộc loại Uncacheable subquery.



EXPLAIN

Partitions
được sử
dụng

Index có thể
sử dụng

Index được
sử dụng

Độ dài key (index
leftmode prefix) được
sử dụng (bytes)

Các cột hoặc const được sử
dụng để lookup giá trị trong
index ở cột key

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	pa	(NULL)	ref	UNIQ	UNIQ	8	const	3	100.00	(NULL)
1	SIMPLE	c	(NULL)	ALL	(NULL)	(NULL)	(NULL)	(NULL)	115228	100.00	Using where; Using join buffer (Block Nested Loop)

type: Join type, Table scan type:

- **const**: So sánh bằng trên cột Primary/Unique Key với 1 hằng số
- **system**: Bảng có duy nhất 1 row
- **eq_ref**: Join/So sánh 1-1 trên cột Primary Key/Unique key với 1 cột ở bảng trước đó. (Duy nhất 1 row)
- **ref**: Join/So sánh dựa trên key (Không phải Unique key, phép join 1-n)
- **ref_or_null**: Giống với ref và kiểm tra thêm điều kiện NULL
- **fulltext**: Join sử dụng full text index
- **index_merge**: Sử dụng index merge
- **unique_subquery**: Thay thế eq_ref với IN subquery
- **index_subquery**: Tương tự unique_subquery với trường hợp subquery sử dụng non-unique indexes.
- **range**: Sử dụng index để range scan bảng
- **ALL**: Full table scan
- **index**: Full index scan



EXPLAIN

Số rows examined phải scan

Estimate % số row còn lại khi filter bởi điều kiện WHERE (Sau quá trình index scan)

- 100%: ko cần filter thêm
- 30%: rows sau filter = $115228 \times 0.3 = 34568$

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	pa	(NULL)	ref	UNIQ	UNIQ	8	const	3	100.00	(NULL)
1	SIMPLE	c	(NULL)	ALL	(NULL)	(NULL)	(NULL)	(NULL)	115228	100.00	Using where; Using join buffer (Block Nested Loop)

Extra: Thông tin bổ sung

- **Using index:** Covering index để hạn chế access vào bảng.
- **Using index condition:** Index Condition Pushdown, sử dụng index để filter trên điều kiện WHERE
- **Using index for group-by:** Như tên gọi.
- **Using where:** Storage engine sử dụng điều kiện WHERE để filter kết quả sau khi sử dụng index
- **Using temporary:** MySQL cần tạo temporary table để xử lý. Nếu kq lớn, việc xử lý có thể flush xuống disk.
- **Using filesort:** external sort, MySQL cần thêm thao tác sort (trên mem hoặc disk)



Chuyên khoa 2: EXPLAIN format=json

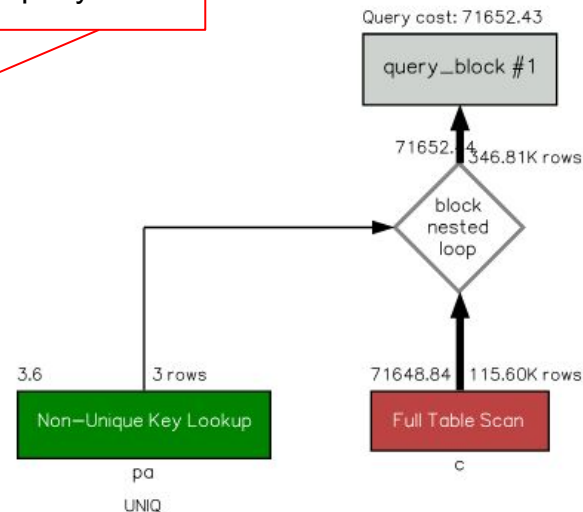
```
EXPLAIN format=json
SELECT pa.`package_order`,
pa.`first_address`, c.`alias`,
c.`first_cod_id`
FROM carts c
INNER JOIN `package_addresses` pa
  ON c.id = pa.cart_id
WHERE pa.package_order =
1510775540 ;
```

join algorithm

query cost

```
{
  "query_block": {
    "select_id": 1,
    "cost_info": {
      "query_cost": "71652.43"
    },
    "nested_loop": [
      {
        "table": {
          "table": "pa"
        }
      },
      {
        "table": {
          "table": "c"
        }
      }
    ]
  }
}
```

table & join order



id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	pa	(NULL)	ref	UNIQ	UNIQ	8	const	3	100.00	(NULL)
1	SIMPLE	c	(NULL)	ALL	(NULL)	(NULL)	(NULL)	(NULL)	115228	100.00	Using where; Using join buffer (Block Nested Loop)



EXPLAIN format=json

possible_keys

ref

used_key_parts
(leftmode prefix)

Cost info

```
{
  "table": {
    "table_name": "pa",
    "access_type": "ref",
    "possible_keys": [
      "UNIQ"
    ],
    "key": "UNIQ",
    "used_key_parts": [
      "package_order"
    ],
    "key_length": "8",
    "ref": [
      "const"
    ],
    "rows_examined_per_scan": 3,
    "rows_produced_per_join": 3,
    "filtered": "100.00",
    "cost_info": {
      "read_cost": "3.00",
      "eval_cost": "0.60",
      "prefix_cost": "3.60",
      "data_read_per_join": "30K"
    },
  },
}
```



EXPLAIN format=json

full table scan?

row examined

row join

select column

Wait a minute...
real condition???

Optimizer hint

```
{
  "table": {
    "table_name": "carts",
    "access_type": "ALL",
    "rows_examined_per_scan": 115603,
    "rows_produced_per_join": 346809,
    "filtered": "100.00",
    "using_join_buffer": "Block Nested Loop",
    "cost_info": {
      "read_cost": "2287.04",
      "eval_cost": "69361.80",
      "prefix_cost": "71652.44",
      "data_read_per_join": "16"
    },
    "used_columns": [
      "id",
      "alias",
      "first cod id"
    ],
    "attached_condition": "(convert(`ghkt`.`c`.`id` using utf8mb4) = `ghkt`.`pa`.`cart id`)",
    "_hint": "Very High - very costly for large tables (not so much for small ones).\\nNo usable indexes were found for the table and the optimizer must search every row.\\nThis could also mean the search range is so broad that the index would be useless.",
    "_access_type": "Full Table Scan"
  }
}
```



EXPLAIN format=json

SHOW WARNINGS;

```
/* select#1 */ select
`ghstk`.`pa`.`package_order` AS
`package_order`,`ghstk`.`pa`.`first_address` AS
`first_address`,`ghstk`.`c`.`alias` AS
`alias`,`ghstk`.`c`.`first_cod_id` AS
`first_cod_id`
from `ghstk`.`package_addresses` `pa`
join `ghstk`.`carts` `c`
where ((`ghstk`.`pa`.`package_order` =
1510775540) and (convert(`ghstk`.`c`.`id` using
utf8mb4) = `ghstk`.`pa`.`cart_id`))
```

Kết luận:

Do cột **cars.id** và **package_addresses.cart_id** khác charset (utf8 vs utf8mb4)

- MySQL muốn so sánh phải convert về cùng utf8mb4.
- wrap function. Không thể sử dụng index.



EXPLAIN format=json



Lợi ích

- Cho chúng ta biết chi tiết kế hoạch thực thi của câu truy vấn.
- Các chỉ số giúp chúng ta hiểu về quá trình thực thi truy vấn và giúp tìm cách tối ưu câu truy vấn.

Hạn chế

- Không nói cho chúng ta biết bản chất vấn đề. Tại sao kế hoạch thực thi đó lại tối ưu và được lựa chọn.
- Không có thông tin về triggers, store functions,...
- Các con số thống kê chỉ là ước lượng và có thể outdate.
- Không nói lên tất cả vấn đề!



Hãy thử với 1 case khác

```
CREATE TABLE `ghtk_user` (
  `id` INT NOT NULL AUTO INCREMENT,
  `username` VARCHAR(30) NOT NULL,
  `fullname` VARCHAR(255) DEFAULT NULL,
  `birthday` DATE NOT NULL,
  `group_id` INT NOT NULL,
  `created` DATETIME DEFAULT
CURRENT TIMESTAMP,
  PRIMARY KEY (`id`),
  KEY `IDX BIRTHDAY USERNAME`
(`birthday`,`username`)
) ENGINE=INNODB
DEFAULT CHARSET=utf8mb3
COLLATE=utf8mb3_unicode_ci;
```

id	username	fullname	birthday	created
1	admin	ADMIN	1995-05-13	2013-06-12 13:33:19
14586	anbh	BÙI HẢI AN	2000-12-24	2017-11-09 09:15:49
54537	anhbm	BÙI MAI ANH	1993-12-04	2018-03-27 08:35:25
2385	anhctv	CHẾ THỊ VÂN ANH	1998-11-20	2017-02-21 10:56:34
10706	anhdc	ĐÀO CÔNG ANH	1993-12-26	2017-10-20 12:02:47
5261	anhdm2	ĐÀM MINH ANH	1999-06-15	2017-07-14 08:29:41
55881	anhdn1	ĐỖ NGỌC ANH	1997-01-29	2018-03-30 17:04:31
3874	anhht4	HỒ TUẤN ANH	1991-08-21	2017-05-29 16:48:07
41493	anhhtm	HOÀNG THỊ MAI ANH	1998-03-27	2018-03-01 08:21:02
8411	anhlv1	LÝ VĂN ANH	1996-05-13	2017-09-25 09:37:18
42821	anhmh	MAI HOÀNG ANH	1993-07-19	2018-03-03 13:31:31
21142	anhmx	MAI XUÂN ANH	1992-11-13	2017-12-05 08:22:45
635	anhnt5	NGUYỄN TUẤN ANH	1989-05-15	2015-09-17 09:19:54
6801	anhnth	NGUYỄN THỊ HỒNG ANH	1999-02-23	2017-08-24 10:08:59
8741	anhpt6	PHẠM THẾ ANH	1993-03-27	2017-10-03 08:27:09

EXPLAIN format=json

```

key": "INX_BIRTHDAY_USERNAME",
  "used key parts": [
    "birthday"
  ],
  "key_length": "95",
  "rows examined per scan": 5005,
  "rows produced per join": 556,
  "filtered": "11.11",
  "index_condition":
    "((`gh tk_user`.`birthday` between
    '1998-01-01' and '1999-12-31') and
    (`gh tk_user`.`username` like 'linh%'))",

```

EXPLAIN

```

SELECT id, username, fullname, birthday
FROM gh tk_user
WHERE birthday BETWEEN '1998-01-01' AND
'1999-12-31' AND username LIKE 'linh%';

```

select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
SIMPLE	gh tk_user	(NULL)	range	IDX_BIRTHDAY_USERNAME	IDX_BIRTHDAY_USERNAME	95	(NULL)	5005	11.11	Using index condition

IDX_BIRTHDAY_USERNAME được sử dụng với key_path: *birthday*

Index condition push down
key_len = 3*30 + 5 = 95

Hãy thử đổi điều kiện birthday bắt đầu từ
"1997-01-01" thay cho "1998-01-01"

Triệu chứng:
Đổi param từ "1998" thành "1997" thì bị
slow query.

```
EXPLAIN
SELECT id, username, fullname, birthday
FROM ghtk_user
WHERE birthday BETWEEN '1997-01-01' AND '1999-12-31' AND username
LIKE 'linh%';
```

id	select_type	table	partitions	type possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	ghtk_user	(NULL)	ALL IDX_BIRTHDAY_USERNAME	(NULL)	(NULL)	(NULL)	32285	4.68	Using where

Tại sao tình huống này không sử dụng **IDX_BIRTHDAY_USERNAME**
mà lại chọn Full table scan???

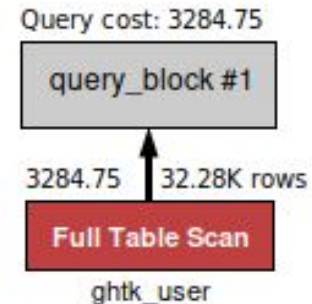


Khám chuyên khoa

SHOW WARNINGS;

-> Không có gì bất thường!

```
/* select#1 */ select `gh tk_user`.`id` AS  
`id`,`gh tk_user`.`username` AS  
`username`,`gh tk_user`.`fullname` AS  
`fullname`,`gh tk_user`.`birthday` AS  
`birthday`  
from `gh tk_user`  
where ((`gh tk_user`.`birthday` between  
'1997-01-01' and '1999-12-31') and  
(`gh tk_user`.`username` like 'linh%'))
```



EXPLAIN format=json

```
"query_block": {  
  "select id": 1,  
  "cost info": {  
    "query_cost": "3284.75"  
  },  
  "table": {  
    "table name": "gh tk user",  
    "access type": "ALL",  
    "possible keys": [  
      "IDX_BIRTHDAY_USERNAME"  
    ],  
    "rows examined per scan": 32285,  
    "rows produced per join": 1510,  
    "filtered": "4.68",  
    ...  
  },  
}
```

Bó tay?

```
...  
"cost_info": {  
  "read cost": "3133.70",  
  "eval_cost": "151.05",  
  "prefix cost": "3284.75",  
  "data_read_per_join": "2M"  
},  
"used columns": [  
  "id",  
  "username",  
  "fullname",  
  "birthday"  
],  
"attached_condition":  
"((`itlinhph`.`gh tk user`.`birthday`  
between '1997-01-01' and '1999-12-31')  
and (`itlinhph`.`gh tk_user`.`username`  
like 'linh%'))"  
}  
}
```



Khám theo yêu cầu: OPTIMIZER TRACE

EXPLAIN: Trả lời câu hỏi HOW, diễn giải kế hoạch thực thi được chọn

- Kế hoạch thực thi câu truy vấn như thế nào?

OPTIMIZER TRACE: Trả lời câu hỏi WHY

- Tại sao kế hoạch đó được chọn, có những kế hoạch nào khác?
- Quá trình thực thi chi tiết và nhiều thông tin khác

```
SET SESSION OPTIMIZER_TRACE="enabled=on"; # enable tracing
```

```
<statement to trace>; # ex: SELECT, EXPLAIN SELECT, UPDATE, DELETE...
```

```
SELECT * FROM information_schema.OPTIMIZER_TRACE; # show tracing info
```

```
SET SESSION OPTIMIZER_TRACE="enabled=off"; # disable tracing
```

OPTIMIZER TRACE

```
  "analyzing_range_alternatives": {
    "range_scan_alternatives": [
      {
        "index": "IDX_BIRTHDAY_USERNAME",
        "ranges": [
          "'1997-01-01' <= birthday <= '1999-12-31'"
        ],
        "index_dives_for_eq_ranges": true,
        "rowid_ordered": false,
        "using_mrr": false,
        "index_only": false,
        "in_memory": 1,
        "rows": 13596,
        "cost": 4758.86,
        "chosen": false,
        "cause": "cost"
      }
    ],
    "analyzing_roworder_intersect": {
      "usable": false,
      "cause": "too_few_roworder_scans"
    }
  }
}
```

Ah hah
Cost full scan bé hơn!
Không sử dụng được
range scan ở cột
username

Số row phải scan ít
hơn tại sao cost lại
cao hơn???

```
  "considered_execution_plans": [
    {
      "plan_prefix": [],
      "table": "`ghik_user`",
      "best_access_path": {
        "considered_access_paths": [
          {
            "rows_to_scan": 32285,
            "access_type": "scan",
            "resulting_rows": 32285,
            "cost": 3284.75,
            "chosen": true
          }
        ],
        "condition_filtering_pct": 100,
        "rows_for_plan": 32285,
        "cost_for_plan": 3284.75,
        "chosen": true
      }
    }
  ],
}
```

Show full optimizer trace result [here](#)



Index

[illegible]

Table

id	username	fullname	birthday
4203	huyhd2	HÀ ĐỨC HUY	1997-03-08
4442	thangnh	NGUYỄN HỮU THĂNG	1997-03-02
5465	khoatv	TRẦN VĂN KHOA	1997-03-28
5612	hungtd3	TỪ ĐỨC HUNG	1997-04-27
6438	phuongnth	NGUYỄN THỊ HỒNG PHƯƠNG	1996-12-26
6875	lypt2	PHAN THỊ LY	1997-02-13
8050	diennm	NGUYỄN MINH DIỄN	1996-12-29
8350	tuyennt	NGUYỄN THANH TUYỀN	1997-04-11
10298	phuongbh	BÙI HỒNG PHƯƠNG	1997-02-03
12690	tamln	LÊ NHỰT TÂM	1997-03-05
24391	binhht1	HỒ THANH BÌNH	1997-02-19
25273	linhnt6	NGUYỄN THẾ LINH	1997-01-25
25465	toantv3	TRẦN VIỆT TOÀN	1997-03-30
30001	danhph	PHÙNG HOÀNG DANH	1997-04-14
35073	tinhdv	DANH VĂN TÍNH	1997-01-03
36089	linhntk	NGUYỄN THỊ KIM LINH	1997-03-26
38853	ngihat1	TẶNG NGHĨA	1996-12-22
39889	hunglt5	LÊ TẤN HÙNG	1997-03-24
40253	linhnv12	NGUYỄN VĂN LINH	1997-05-01

Read
block
16kb

vs

10
cost
thấp
hơn

id	username	fullname	birthday
4203	huyhd2	HÀ ĐỨC HUY	1997-03-08
4442	thangnh	NGUYỄN HỮU THĂNG	1997-03-02
5465	khoatv	TRẦN VĂN KHOA	1997-03-28
5612	hungtd3	TỬ ĐỨC HUNG	1997-04-27
6438	phuongnth	NGUYỄN THỊ HỒNG PHƯƠNG	1996-12-26
6875	lypt2	PHAN THỊ LY	1997-02-13
8050	diennm	NGUYỄN MINH DIỄN	1996-12-29
8350	tuyennt	NGUYỄN THANH TUYỀN	1997-04-11
10298	phuongbh	BÙI HỒNG PHƯƠNG	1997-02-03
12690	tamln	LÊ NHỰT TÂM	1997-03-05
24391	binhht1	HỒ THANH BÌNH	1997-02-19
25273	linhnt6	NGUYỄN THẾ LINH	1997-01-25
25465	toantv3	TRẦN VIỆT TOÀN	1997-03-30
30001	danhph	PHÙNG HOÀNG DANH	1997-04-14
35073	tinhdv	DANH VĂN TỈNH	1997-01-03
36089	linhntk	NGUYỄN THỊ KIM LINH	1997-03-26
38853	ngihat1	TẶNG NGHĨA	1996-12-22
39889	hunglt5	LÊ TẤN HÙNG	1997-03-24
40253	linhnv12	NGUYỄN VĂN LINH	1997-05-01

Sửa 1 chút cấu trúc dữ liệu và ảo thuật:
Thử đánh index (username, birthday)

```
ALTER TABLE `gh tk_user`  
DROP INDEX `IDX_BIRTHDAY_USERNAME`,  
ADD INDEX `IDX_USERNAME_BIRTHDAY` (`username`, `birthday`);
```

Covering index???

```
SELECT id, username, birthday  
FROM gh tk_user  
WHERE username LIKE 'linh%'  
AND birthday BETWEEN '1997-01-01' AND  
'1999-12-31';
```

```
"potential_range_indexes": [  
  {  
    "index": "PRIMARY",  
    "usable": false,  
    "cause": "not_applicable"  
  },  
  {  
    "index": "INX_USERNAME_BIRTHDAY",  
    "usable": true,  
    "key_parts": [  
      "username",  
      "birthday",  
      "id"  
    ]  
  }  
]
```

select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
SIMPLE	gh tk_user	(NULL)	range	IDX_USERNAME_BIRTHDAY, IDX_USERNAME	IDX_USERNAME_BIRTHDAY	92	(NULL)	386	42.11	Using index; Using where



Tổng kết các công cụ chẩn đoán



QUERY COST

- Query runtime
- Row sent, Row examined
- Query cost (IO, CPU)
- Chẩn đoán lâm sàng

EXPLAIN

- Nâng cao: EXPLAIN format=json
- Hiểu về kế hoạch thực thi của câu truy vấn

OPTIMIZER TRACE

- Hiểu chi tiết tại sao database lại chọn kế hoạch thực thi đó.
- Các bước thực thi và lựa chọn kế hoạch truy vấn.

3

KÊ ĐƠN



Tối ưu truy vấn SELECT



**Tối ưu truy vấn MODIFY
(Insert, Update, Delete,...)**



Các chiến lược khác

01

WHERE CONDITION

✗ `WHERE (b>=5 AND b=5) OR (b=6 AND 5=5) OR (b=7 AND 5=6)`

✓ `WHERE b=5 OR b=6`

✗ `WHERE ((a AND b) AND c OR (((a AND b) AND (c AND d))))`

✓ `WHERE (a AND b AND c) OR (a AND b AND c AND d)`

✗ `WHERE YEAR(birthday) + 2 > YEAR('1999-12-01')`

✓ `WHERE birthday > YEAR('1999-12-01') - 2`

✓ Leftmode prefix

✓ Index (const, range)

✓ Multi-column index

✓ Index condition pushdown

✓ Index Merge



Đơn giản hóa logic

- Bộ optimizer không phải bao giờ cũng hiệu quả. Hãy biến đổi logic về đơn giản nhất.
- Chuyển các tính toán logic sang vế phải



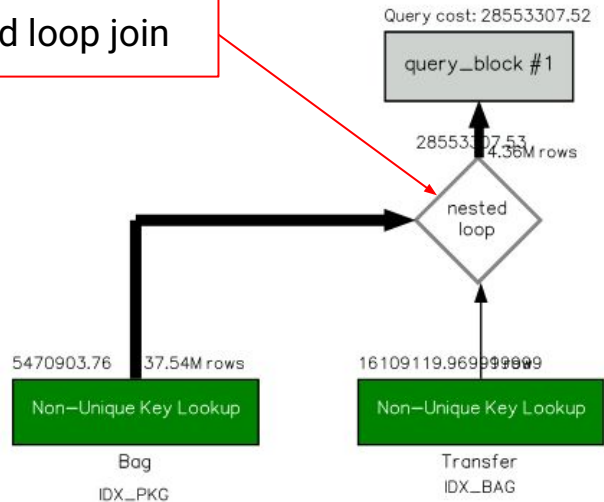
Đánh index hiệu quả

- Áp dụng các quy tắc đánh index đã học ở lesson 2 để tối ưu việc sử dụng index trong điều kiện WHERE.

02

JOIN

Nested loop join



INNER JOIN vs LEFT JOIN

- Chỉ định thứ tự Join bằng LEFT JOIN
- Chú ý nếu LEFT JOIN có mệnh đề WHERE trên bảng bên phải thì LEFT JOIN ~ INNER JOIN

```
SELECT * FROM t1 LEFT JOIN t2 ON (column1)
WHERE t2.column2=5;
```



NESTED LOOP JOIN

- Thuật toán join được implement ở hầu hết database.
- Đặt bảng có điều kiện filter tốt sang trái.
- Note: MySQL 5.7 trở về trước chỉ hỗ trợ nested loop join.

02

JOIN

```
SELECT pa.`package_order`,  
pa.`first address`, c.`alias`,  
c.`first_cod_id`  
FROM `package addresses` pa  
INNER JOIN carts c ON c.id = pa.cart_id  
WHERE pa.package_order = 1510775540 ;
```



```
SELECT pa.`cart_id`, pa.`package_order`,  
pa.`first address` FROM  
package addresses pa WHERE  
pa.`package_order` = 1510775540;
```

```
SELECT c.`alias`, c.`first_cod_id` FROM  
carts c WHERE c.id IN  
("636a19e6-cc48-4c35-ba38-431b0a6e4d78",  
"5f06984d-1fe8-429a-b1e9-4b160a0a3422");
```



Đánh index với cột kết nối trong mệnh đề ON

- Đánh index với cột kết nối 2 bảng.
- Chú ý thứ tự JOIN. Nếu bảng A JOIN B qua cột $A.col1 = B.col2$ thì chỉ cần đánh index trên cột B.col2



Buông bỏ JOIN (JOIN Decomposition)

- Tách nhỏ các câu truy vấn đầy join về tăng application.
- Tận dụng cache, tránh lock, tránh các case khác charset.

03

COUNT()

EXPLAIN

rows	filtered	Extra
32285	4.68	Using where

$$\text{estimated row} = 32285 * 4.68\% = 1510$$
**Sử dụng COUNT(*) thay COUNT(col)**

- Nếu muốn count số records trả về hãy dùng count(*) để tránh nhập nhầm NULL value

**Tính xấp xỉ**

- Sử dụng EXPLAIN để tính xấp xỉ số row trả về
- SHOW TABLE STATUS estimate số row của bảng

**Kỹ thuật Caching**

- Sử dụng redis, memcached, ... như 1 external cache
- Build bảng tạm, ...

04

ORDER BY, GROUP BY

**Sử dụng index cho các toán tử ORDER BY, GROUP BY**

- Đánh index với cột trong mệnh đề ORDER BY, GROUP BY để hạn chế SORTING
- Với MySQL 8.0 đánh Descending Indexes cho các toán tử ORDER BY ASC, DESC custom.

**ORDER BY NULL**

- Trong toán tử GROUP BY nếu không cần sắp xếp kết quả, hãy thêm ORDER BY NULL để hạn chế SORTING (Từ MySQL 5.7 trở về trước)

**Sử dụng các kỹ thuật khác**

- Bản thân các database dạng OLTP không thiết kế phù hợp cho các câu truy vấn tổng hợp, thống kê.
- Hãy sử dụng database OLAP, hoặc tính toán trước kết quả qua các kỹ thuật CDC, tmp table,..

05 LIMIT, OFFSET

✗ `SELECT id, username, fullname
FROM users LIMIT 10 OFFSET 1000;`

✓ `SELECT id, username, fullname
FROM users WHERE id >= 1000 AND id <
1010;`

✓ `SELECT id, username, fullname
FROM users
ORDER BY id LIMIT 10 # last id = 123;`

`SELECT id, username, fullname
FROM users WHERE id > 123
ORDER BY LIMIT 10; # Next page start
from 123;`



LIMIT dữ liệu trả về

- Nếu không cần phải lấy toàn bộ dữ liệu, hãy sử dụng LIMIT để hạn chế số records trả về.
- Không SELECT *, chỉ select các cột cần thiết.



Cẩn thận với LIMIT OFFSET

- Câu truy vấn LIMIT 10 OFFSET 10000 tương đương với việc database phải truy cập qua 10010 records để lấy 10 records cuối cùng!
- Với các màn hình phân trang, các màn cuối thường sẽ rất chậm nếu sử dụng LIMIT OFFSET. Chuyển qua phân trang WHERE trên Key



Tách truy vấn

- Nếu kết quả LIMIT phụ thuộc 1 phần vào 1 bảng rất lớn, hãy tách thành các câu truy vấn con, LIMIT OFFSET 1 phần trên bảng lớn sau đó mới join tới các bảng khác.

06

UNION



```
(SELECT username, fullname
  FROM users ORDER BY username)
UNION
  (SELECT username, fullname
   FROM ighk_users ORDER BY
username)
LIMIT 20;
```



```
(SELECT username, fullname
  FROM users ORDER BY username
 LIMIT 20)
UNION ALL
  (SELECT username, fullname
   FROM ighk_users ORDER BY
username
  LIMIT 20)
LIMIT 20;
```



UNION

- MySQL mặc định sẽ tạo bảng tạm (mem hoặc disk) để gộp kết quả của các câu truy vấn. Chi phí cho UNION khá lớn.
- Hãy **LIMIT** kết quả trả về nhất có thể trước khi UNION



UNION vs UNION ALL

- Nếu không cần distinct kết quả trả về, hãy sử dụng **UNION ALL** để tránh MySQL phải thêm thao tác so sánh, loại bỏ giá trị trùng lặp.

07

DISTINCT, SUBQUERY

```
SELECT DISTINCT (stations.name)
FROM packages p INNER JOIN stations s
ON (p.station_id = s.id);
```



```
SELECT id, name
FROM stations
WHERE EXISTS (SELECT * FROM packages
WHERE stations.id =
packages.station_id);
```

**Sử dụng EXISTS để tránh DISTINCT**

- Trong một số trường hợp cần *DISTINCT* kết quả, có thể chuyển qua dùng *EXISTS* để tránh việc phải tạo bảng tạm.

**Convert Subquery**

- Với MySQL 5.7 trở về trước, convert subquery *WHERE IN (SELECT ... FROM ...)* sang *EXISTS* hoặc chuyển về *JOIN*.
- Hạn chế dependency subquery

08

MODIFY DATA (INSERT, UPDATE, DELETE,...)

Thời gian INSERT/MODIFY 1 record:

- Connecting: (3)
- Sending query to server: (2)
- Parsing query: (2)
- Inserting row: (1 × size of row)
- Inserting indexes: (1 × number of indexes)
- Closing: (1)



INSERT/UPDATE/DELETE theo batch

- Cập nhật dữ liệu theo batch giúp giảm IO, CPU, giảm thời gian cập nhật.
- Quá trình MODIFY data thực hiện qua nhiều bước, nó còn phức tạp hơn khi sử dụng transaction và trong hệ thống có replication.



LIMIT DELETED RECORDS

- Sử dụng option **LIMIT** để giới hạn số lượng bản ghi được xóa trong 1 transaction.



TRUNCATE TABLE

- Nếu muốn xóa nhanh toàn bộ dữ liệu trong bảng. Sử dụng **TRUNCATE TABLE** thay **DELETE**.
- Cần thận khi sử dụng trên production vì thao tác này không ghi lại binlog thay đổi.

08

MODIFY DATA (INSERT, UPDATE, DELETE,...)

```
DELETE FROM packages
WHERE created < NOW() - INTERVAL 1
MONTH;
```



```
rows_affected = 0
do {
  rows affected = do query(
    "DELETE FROM packages WHERE
    created < NOW() - INTERVAL 1
    MONTH LIMIT 5000")
} while rows_affected > 0
```

**INSERT/UPDATE/DELETE theo batch**

- Cập nhật dữ liệu theo batch giúp giảm IO, CPU, giảm thời gian cập nhật.
- Quá trình MODIFY data thực hiện qua nhiều bước, nó còn phức tạp hơn khi sử dụng transaction và trong hệ thống có replication.

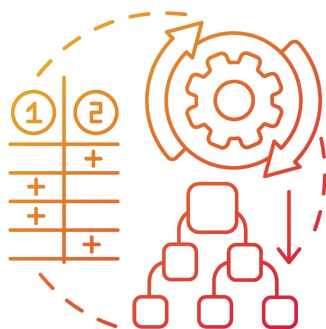
**LIMIT DELETED RECORDS**

- Sử dụng option **LIMIT** để giới hạn số lượng bản ghi được xóa trong 1 transaction.

**TRUNCATE TABLE**

- Nếu muốn xóa nhanh toàn bộ dữ liệu trong bảng. Sử dụng **TRUNCATE TABLE** thay **DELETE**.
- Cần thận khi sử dụng trên production vì thao tác này không ghi lại binlog thay đổi.

09 RESTRUCTURE QUERY



DIVIDE AND CONQUER



Chia truy vấn phức tạp thành các câu truy vấn con đơn giản.

- MySQL hay các database OLTP nói chung được thiết kế phù hợp với việc handle nhiều câu truy vấn nhỏ, response nhanh.
- Chia nhiều truy vấn có thể tận dụng xử lý song song



Cắt nhỏ câu truy vấn

- Với các câu **WHERE IN** lớn cần cắt nhỏ thành các batch để truy vấn (Hạn chế full scan bảng, transaction lớn, copy list qua từng lần JOIN)
- **INSERT/UPDATE/DELETE** theo batch đủ nhỏ để hạn chế replication lag.



Chuyển xử lý logic về tầng application

- Không handle và xử lý logic bằng query.
- Scale ở tầng application dễ hơn rất nhiều so với scale tầng database.

10

OPTIMIZER HINT

```
SELECT /*+ NO RANGE OPTIMIZATION(t3  
PRIMARY, f2_idx) */ f1  
FROM t3 WHERE f1 > 30 AND f1 < 33;
```

```
SELECT  
/*+ MAX EXECUTION_TIME(1000) */  
FROM packages;
```

```
SELECT * FROM new_package_logs USE  
INDEX (IDX_PACKAGE)  
WHERE pkg_order=1932432423 AND  
action="Transit";
```



OPTIMIZER HINT

- Optimizer không phải lúc nào cũng đi đúng hướng.
- Đưa ra 1 số hint cho Optimizer tránh làm đường lạc lối:
 - Join order
 - Use index, ignore index
 - Range scan, index merge,...



FORCE INDEX

- Với một số trường hợp có thể đưa query bắt buộc sử dụng 1 index được chỉ định.

TỔNG KẾT



Cần hiểu rõ kiến trúc xử lý câu truy vấn trước khi thực hiện tối ưu.



Không có một khuôn mẫu nào để tối ưu câu truy vấn. Hướng xử lý phải tùy bài toán và tình huống.



**Tối ưu truy vấn là một môn học thực hành.
Hãy kiên nhẫn thử nghiệm.**

FEEDBACK KHÓA HỌC





LightUP
Your Potential

CHƯƠNG TRÌNH ĐÀO TẠO DOMAIN BACKEND

TRÂN TRỌNG CẢM ƠN

Chúc bạn có trải nghiệm học tập và làm việc tốt tại GHK

